

OWASP Top 10 2025

Application Design Flaws — Complete Room Writeup

Platform	TryHackMe
Room	OWASP Top 10 2025: Application Design Flaws
Tasks	AS02 · AS03 · AS04 · AS06

Set up your virtual environment

To successfully complete this room, you'll need to set up your virtual environment. This involves starting both your AttackBox (if you're not using your VPN) and Target Machines, ensuring you're equipped with the necessary tools and access to tackle the challenges ahead.



Attacker machine ⓘ

Status: ● Off

[Start AttackBox](#) ▼



Target machine ⓘ

Status: ● Off

[▶ Start Machine](#)



This room breaks each 4 of the OWASP Top 10 2025 categories. In this room, you will learn about the categories that are related to failures in architecture and system design. You will put the theory into practice by completing supporting challenges. The following categories are covered in this room:

1. AS02: Security Misconfigurations
2. AS03: Software Supply Chain Failures
3. AS04: Cryptographic Failures
4. AS06: Insecure Design

Deploy Practical

Before we begin, please deploy the practical VM by clicking the green "Start Machine" button at the start of this task. Please note that you will need to use either the TryHackMe AttackBox or your own hacking machine connected to the TryHackMe VPN to access each practical.

Task 2 — AS02: Security Misconfigurations

What is Security Misconfiguration?

Security misconfigurations happen when systems are deployed with **unsafe defaults, incomplete settings, or exposed services**. These are not code bugs — they are mistakes in how the environment is set up. A single verbose error message can expose sensitive data or credentials to an attacker.

Task 2 AS02: Security Misconfigurations

Security Misconfigurations

What It Is

Security misconfigurations happen when systems, servers, or applications are deployed with unsafe defaults, incomplete settings, or exposed services. These are not code bugs but mistakes in how the environment, software, or network is set up. They create easy entry points for attackers.

Why It Matters

Even small misconfigurations can expose sensitive data, enable privilege escalation, or give attackers a foothold into the system. Modern applications rely on complex stacks, cloud services, and third-party APIs. A single exposed admin panel, an open storage bucket, or misconfigured permissions can compromise the entire system.

Example

In 2017, [Uber](#) exposed a backup AWS S3 bucket with sensitive user data, including driver and rider information, because the bucket was publicly accessible. Attackers could download data directly without needing credentials. This shows how a deployment mistake can lead to a significant breach.

Common Patterns

- Default credentials or weak passwords left unchanged
- Unnecessary services or endpoints exposed to the internet
- Misconfigured cloud storage or permissions (S3, Azure Blob, GCP buckets)
- Unrestricted API access or missing authentication/authorisation
- Verbose error messages exposing stack traces or system details
- Outdated software, frameworks, or containers with known vulnerabilities
- Exposed AI/ML endpoints without proper access controls

How To Prevent It

- Harden default configurations and remove unused features or services
- Enforce strong authentication and least privilege across all systems
- Limit network exposure and segment sensitive resources
- Keep software, frameworks, and containers up to date with patches
- Hide stack traces and system information from error messages
- Audit cloud configurations and permissions regularly
- Secure AI endpoints and automation services with proper access controls and monitoring
- Integrate configuration reviews and automated security checks into your deployment pipeline

Challenge

Navigate to `MACHINE_IP:5002`. It appears that the developers left too many traces in their User Management APIs.

AS02 task description and challenge instructions

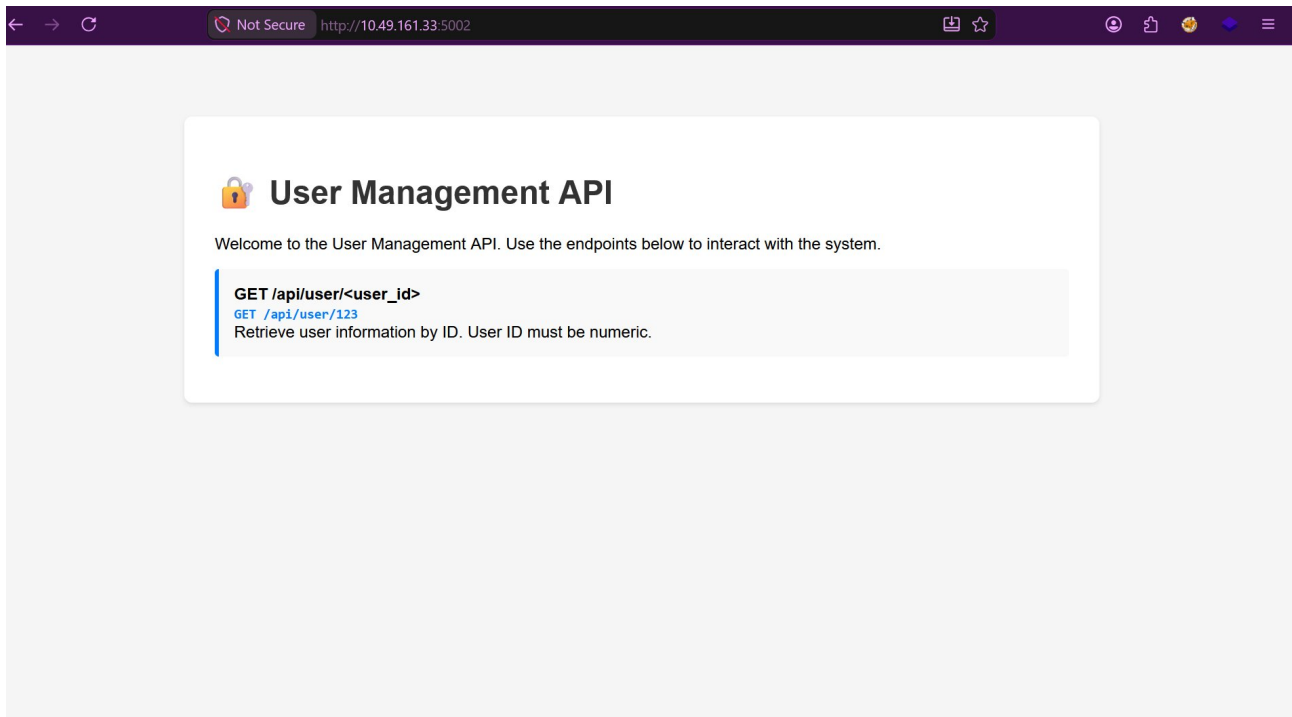
The Challenge

Navigate to `MACHINE_IP:5002`. The developers left too many traces in their User Management APIs. Trigger an error that leaks the flag.

Step-by-Step Solution

Step 1 — Open the API homepage

Go to `http://MACHINE_IP:5002`. You see a User Management API with one documented endpoint: `GET /api/user/<user_id>`. The example shown is `/api/user/123`.



User Management API homepage at port 5002

Step 2 — Test with a valid numeric ID using Burp Suite

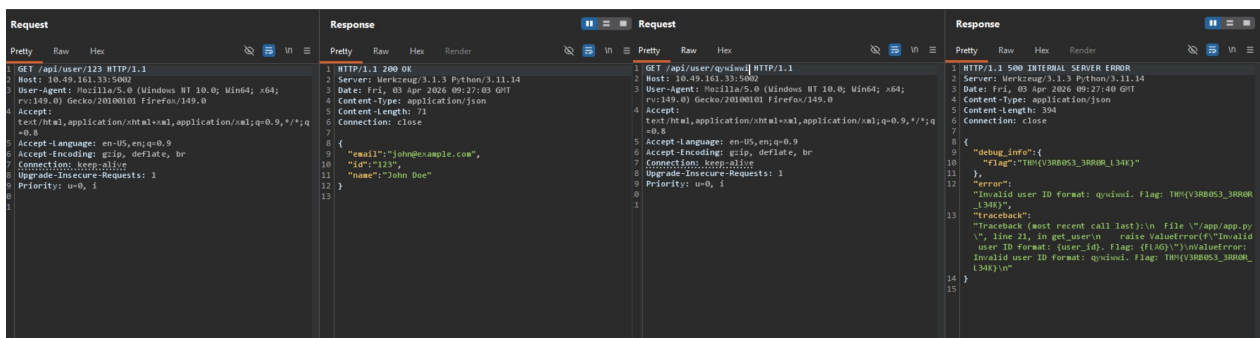
Intercept the request in Burp Suite and send `GET /api/user/123` to Repeater. The server returns 200 OK with normal JSON: name, email, and id. This confirms the endpoint works for valid input.

Step 3 — Send an invalid non-numeric ID

In Burp Repeater, change the user ID to a random non-numeric string like `qywiwwi` → `GET /api/user/qywiwwi` and hit Send.

Step 4 — Read the leaked error response

The server throws a **500 Internal Server Error**. Because debug mode is enabled in production, the full Python traceback is returned — and the FLAG is hard-coded inside the `debug_info` field and error message!



Valid request `/api/user/123` → 200 OK with user data

Invalid ID triggers 500 error leaking the flag in `debug_info`

■ **Flag:** `THM{V3RB0S3 3RR0R L34K}`

■ **Why it happened:** The app ran with `debug=True` in production and the developer embedded the FLAG variable inside the error message. Always disable debug mode and strip internal details from error responses before going live.

Task 3 — AS03: Software Supply Chain Failures

What is a Supply Chain Failure?

Supply chain failures happen when apps rely on **outdated, compromised, or unverified third-party components**. The 2021 SolarWinds Orion attack is the real-world example — malicious code was injected into a trusted software update, silently compromising thousands of organisations.

Task 3 AS03: Software Supply Chain Failures

Software Supply Chain Failures

Download Task Files

What It Is

Software supply chain failures happen when applications rely on components, libraries, services, or models that are compromised, outdated, or improperly verified. These weaknesses are not inherent in your code, but rather in the software and tools you depend on. Attackers exploit these weak links to inject malicious code, bypass security, or steal sensitive data.

Why It Matters

Modern applications are built from many third-party packages, APIs, and AI models. One compromised dependency can compromise your entire system, allowing attackers to gain access without ever touching your own code. Supply chain attacks can be automated and distributed, making them hard to detect and very damaging.

Example

In 2021, the [SolarWinds Orion](#) compromise showed the danger of supply chain attacks. Attackers inserted malicious code into a trusted update, affecting thousands of organisations that automatically installed it. This wasn't a bug in SolarWinds' core logic. It was a flaw in the software update building, verification, and distribution process.

With AI, we can observe this when using unverified third-party models or fine-tuned datasets that can embed hidden behaviours, backdoors, or biased outputs, compromising systems or leaking data.

Common Patterns

- Using unverified or unmaintained libraries and dependencies
- Automatically installing updates without verification
- Over-reliance on third-party AI models without monitoring or auditing
- Insecure build pipelines or CI/CD processes that allow tampering
- Poor license or provenance tracking for components
- Lack of monitoring for vulnerabilities in dependencies after deployment

How To Protect The Supply Chain

- Verify all third-party components, libraries, and AI models before use
- Monitor and patch dependencies regularly
- Sign, verify, and audit software updates and packages
- Lock down CI/CD pipelines and build processes to prevent tampering
- Track provenance and licensing for all dependencies
- Implement runtime monitoring for unusual behaviour from dependencies or AI components
- Integrate supply chain threat modelling into the SDLC, including testing, deployment, and update workflows

Challenge

Navigate to `MACHINE_IP:5003`. The code is outdated and imports an old `lib/vulnerable_utils.py` component. Can you *debug* it?

AS03 task description and challenge instructions

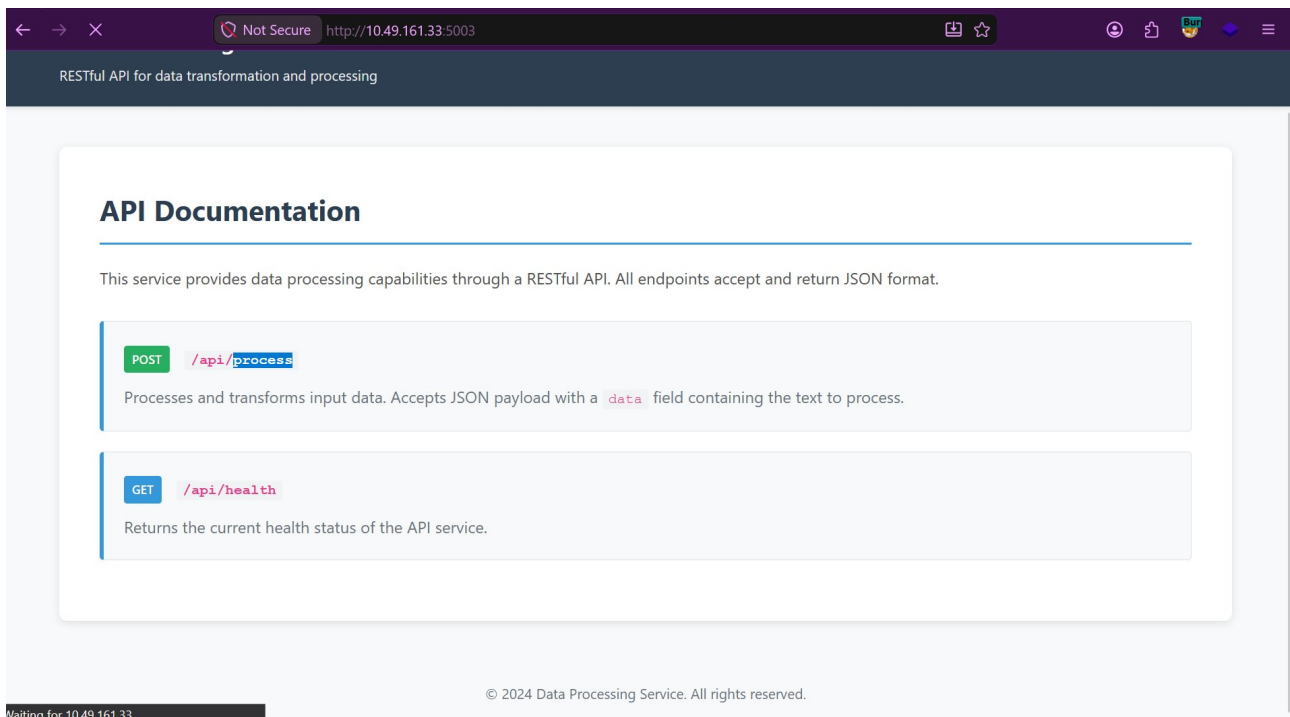
The Challenge

Navigate to `MACHINE_IP:5003`. The code imports an old `lib/vulnerable_utils.py` component. Debug it to find the flag.

Step-by-Step Solution

Step 1 — Open the API documentation

Go to `http://MACHINE_IP:5003`. It is a RESTful data processing API. Two endpoints are listed: `POST /api/process` and `GET /api/health`.



Port 5003 API documentation showing /api/process endpoint

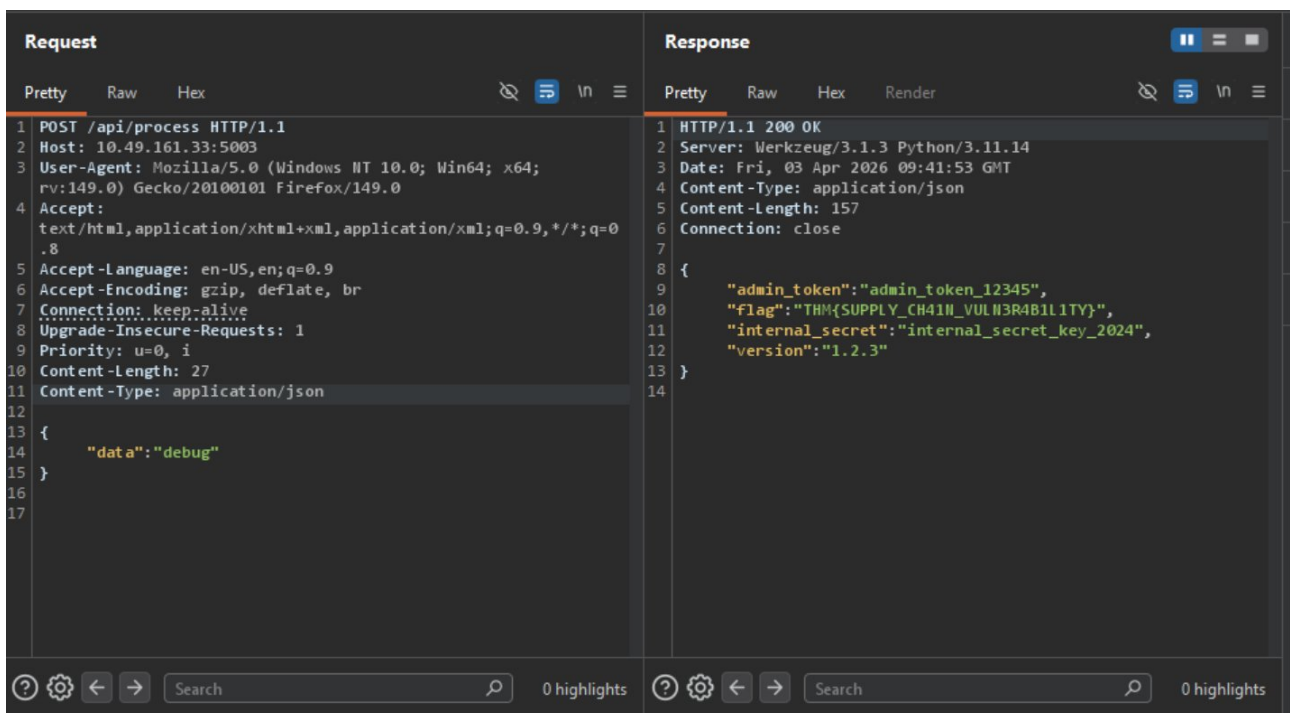
Step 2 — POST the debug keyword via Burp Suite

In Burp Suite Repeater, send a POST to `http://MACHINE_IP:5003/api/process` with Content-Type: `application/json` and this body:

```
{"data": "debug"}
```

Step 3 — Read the leaked secrets in the response

The old vulnerable library has a hidden debug backdoor. Passing 'debug' dumps all internal secrets: the flag, admin token, internal secret key, and version number.



Response leaking flag, admin_token and internal_secret via the debug backdoor

■ Flag: THM{SUPPLY CH41N VULN3R4B1L1TY}

■ Why it happened: An outdated library (vulnerable_utils.py) had a hidden debug backdoor that was never audited. Always verify, pin, and regularly audit all third-party components before and after deployment.

Task 4 — AS04: Cryptographic Failures

What are Cryptographic Failures?

Cryptographic failures occur when encryption is **used incorrectly or not at all** — weak algorithms (MD5, ECB mode), hard-coded keys, or secrets stored in client-side code. Attackers can read data that should be private.

Task 4 🟢 AS04: Cryptographic Failures

Cryptographic Failures

What It Is

Cryptographic failures happen when encryption is used incorrectly or not at all. This includes weak algorithms, hard-coded keys, poor key handling, or unencrypted sensitive data. These flaws let attackers access information that should be private.

Why It Matters

Web applications rely on cryptography everywhere: protecting network traffic, securing stored data, verifying identities, and safeguarding secrets. When these controls fail, sensitive data such as passwords, tokens, or personal information can be exposed, leading to account takeovers or full-scale breaches.

Attackers can exploit these flaws through man-in-the-middle attacks, brute-force attacks on weak keys, or by simply discovering secrets that were never properly protected.

Common Patterns

- Using deprecated or weak algorithms like MD5, SHA-1, or ECB mode
- Hard-coded secrets in code or configuration
- Poor key rotation or management practices
- Lack of encryption for sensitive data at rest or in transit
- Self-signed or invalid TLS certificates
- Using AI/ML systems without proper secret handling for model parameters or sensitive inputs

How To Prevent It

- Use strong, modern algorithms such as AES-GCM, ChaCha20-Poly1305, or enforce TLS 1.3 with valid certificates
- Use secure key management services like Azure Key Vault, AWS KMS, or HashiCorp Vault
- Rotate secrets and keys regularly, following defined crypto periods
- Document and enforce policies and standard operating procedures for key lifecycle management
- Maintain a complete inventory of certificates, keys, and their owners
- Ensure AI models and automation agents never expose unencrypted secrets or sensitive data
- The web application in this room contains a weakness of this type for you to explore.

Challenge

Navigate to MACHINE_IP:5004. Can you find the key to decrypt the file?

Answer the questions below

What's the flag?

THM{CRYPTO_FAILURE_H4RDCOD3D_K3Y}

✓ Correct Answer

AS04 task description — Cryptographic Failures challenge

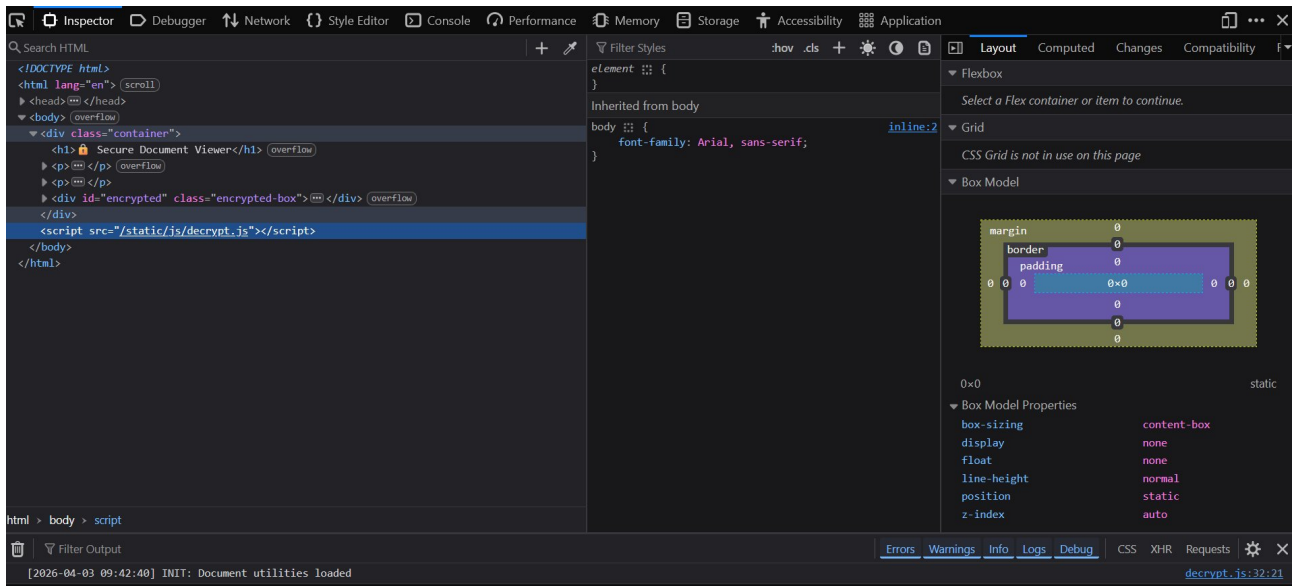
The Challenge

Navigate to MACHINE_IP:5004. Find the key to decrypt the file.

Step-by-Step Solution

Step 1 — Inspect the page source with DevTools (F12)

Open http://MACHINE_IP:5004. Press F12 → Inspector tab. In the HTML you find a script tag referencing /static/js/decrypt.js. Click on it.



Firefox DevTools Inspector showing the decrypt.js script tag in the HTML source

Step 2 — Find the hard-coded secret key in decrypt.js

Inside decrypt.js the encryption configuration is exposed in plain text at the top of the file — readable by anyone visiting the site:

```
// Document encryption utilities
// Last updated: 2024-03-15

(function() {
  'use strict';

  // Configuration
  const SECRET_KEY = "my-secret-key-16";
  const ENCRYPTION_MODE = "ECB";
  const KEY_SIZE = 128;

  // Utility functions
  function formatTimestamp(date) {
    return date.toISOString().replace('T', ' ').substring(0, 19);
  }

  function validateDocumentId(id) {
    return /^[a-zA-Z0-9-_.]+$/.test(id);
  }

  function getDocumentMetadata() {
    return {
      version: "1.2.3",
      encryptionEnabled: false,
      lastModified: formatTimestamp(new Date())
    };
  }

  // Logging utility
  function logEvent(eventType, details) {
    if (typeof console !== 'undefined' && console.log) {
      console.log(`[${formatTimestamp(new Date())}] ${eventType}:`, details);
    }
  }

  // Document validation
  function isValidDocumentFormat(data) {
    try {
      const decoded = atob(data);
      return decoded.length > 0 && decoded.length % 16 === 0;
    } catch (e) {
      return false;
    }
  }
})
```

decrypt.js revealing SECRET_KEY = 'my-secret-key-16' and ENCRYPTION_MODE = 'ECB' hard-coded

```
const SECRET_KEY = 'my-secret-key-16'; const ENCRYPTION_MODE = 'ECB'; const KEY_SIZE = 128;
```

Step 3 — Decrypt using CyberChef

Copy the Base64 ciphertext from the page. Open CyberChef and build this two-step recipe:

Step	Operation	Settings
1	From Base64	Alphabet: A-Za-z0-9+/=
2	AES Decrypt	Key: my-secret-key-16 Mode: ECB Input/Output: Raw

Step 4 — Hit BAKE — flag appears in the output

The screenshot shows the CyberChef interface. On the left is the 'Operations' list. The 'Recipe' panel in the center contains two operations: 'From Base64' and 'AES Decrypt'. The 'From Base64' operation has the alphabet 'A-Za-z0-9+/' and 'Remove non-alphabet chars' checked. The 'AES Decrypt' operation has a key of 'my-secret-key-16', mode 'ECB', and both input and output set to 'Raw'. At the bottom of the recipe panel is a green 'BAKE!' button. The 'Input' panel on the right contains a long Base64-encoded string. The 'Output' panel on the right shows the decrypted result: 'CONFIDENTIAL: The admin password is 'admin123'. Flag: THM{CRYPTO_FAILURE_H4RDCOD3D_K3Y}'.

CyberChef decrypting the ciphertext with AES ECB — flag visible in output panel

Flag: THM{CRYPTO_FAILURE_H4RDCOD3D_K3Y}

Why it happened: The AES key was hard-coded inside a publicly served JavaScript file. ECB mode was also used, which is cryptographically weak. Keys must always be stored server-side in a secrets manager — never in client-side code.

Task 5 — AS06: Insecure Design

What is Insecure Design?

Insecure design means **flawed logic or architecture is baked in from the start**. You cannot patch it — you must rethink the whole system. Clubhouse's early API is a classic example: it had no authentication because the backend assumed only the mobile app would ever call it.

Insecure Design

What It Is

Insecure design happens when flawed logic or architecture is built into a system from the start. These flaws stem from skipped threat modelling, no design requirements or reviews, or accidental errors.

Moreover, with the introduction of AI assistants, AI systems exacerbate insecure design. Developers often assume that models are safe, correct, or predictable, or that the code they produce is flaw-free. When an AI system can generate queries, write code, or classify users without limits, the risk is built into the design, leading to poor architectural patterns.

Example

A good example is [Clubhouse](#). Its early design assumed users would only interact through the mobile app, but the backend API had no proper authentication. Anyone could query user data, room info, and even private conversations directly. When researchers tested it, "the entire "private c "nversation" premise fell apart.

Why It Matters

You can't patch an insecure design. It's built into the workflow, logic, and trust boundaries. Fixing it means rethinking how systems, and now AI, make decisions.

Common Insecure Designs In 2025

- Weak business logic controls, like recovery or approval flows
- Flawed assumptions about user or model behaviour
- AI components with unchecked authority or access
- Missing guardrails for LLMs and automation agents
- Test or debug bypasses left in production
- No consistent abuse-case review or AI threat modelling

Insecure Design In The AI Era

AI introduces new kinds of design failures. For example, prompt injection occurs when user input is blended with system prompts, allowing attackers to hijack the context or extract hidden data. Blind trust in model output creates fragile systems that act on AI decisions without validation or oversight, which is why human review remains necessary. When it comes to poisoned models, pulled from unverified sources or fine-tuned on unsafe data, they can embed hidden behaviours or backdoors that compromise the system from within.

How To Design Securely

- Treat every model as untrusted until proven otherwise.
- Validate and filter all model inputs and outputs to ensure accuracy and integrity.
- Separate system prompts from user content.
- Keep sensitive data out of prompts unless absolutely needed and protect it with strict controls.
- Require human review for high-risk AI actions.
- Log model provenance, monitor behaviour, and apply differential privacy for sensitive data.
- Include AI-specific threat modelling for prompt attacks, inference risks, agent misuse, and supply chain compromise throughout the design process.
- Build threat modelling into every stage of development, not just at the start.

AS06 task description — Insecure Design challenge

The Challenge

Navigate to **MACHINE_IP:5005**. The API has no access controls. Explore the endpoints to find the flag.

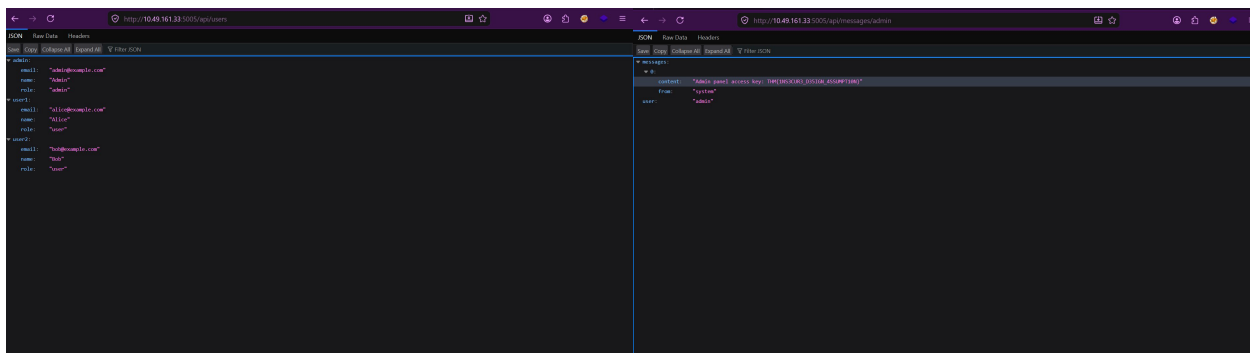
Step-by-Step Solution

Step 1 — Hit /api/users — no login required

Go to `http://MACHINE_IP:5005/api/users`. With zero authentication the endpoint returns all user accounts — including the admin (`admin@example.com`). This reveals the username **admin**.

Step 2 — Access /api/messages/admin — still no auth

Navigate to `http://MACHINE_IP:5005/api/messages/admin`. No login, no token — the API returns the admin's private messages.



/api/users — all accounts exposed with no authentication

/api/messages/admin — system message contains the flag

■ **Flag:** THM{1NS3CUR3 D35IGN 4SSUMPT10N}

■ **Why it happened:** The API was designed with the assumption that only authorised clients would call it. There was no authentication middleware, no RBAC, and no rate limiting. Security must be designed in from day one — it cannot be patched on afterward.

Conclusion & Flag Summary

Task 2 ✓ AS02: Security Misconfigurations

Task 3 ✓ AS03: Software Supply Chain Failures

Task 4 ✓ AS04: Cryptographic Failures

Task 5 ✓ AS06: Insecure Design

Task 6 ✓ Conclusion

Conclusion

Security design failures across AS02 Security Misconfigurations, AS03 Software Supply Chain Failures, AS04 Cryptographic Failures, and AS06 Insecure Design all come from the same root cause: weak foundations. You cannot add security at the end and expect it to work. Strong systems start with clear security requirements, realistic threat assumptions, controlled configurations, vetted dependencies, and sound cryptographic choices.

Treat defaults with suspicion, treat every dependency as a potential risk, and keep design simple enough to reason about. Get the design right early, and you avoid a long future of preventable incidents.

Continue the journey with **Room 3 in this module: Application Design Flaws**: <https://tryhackme.com/jr/owasptop102025insecuredatahandling>

Answer the questions below

I'm ready for the next room!

No answer needed

✓ Correct Answer

All four tasks completed — green checkmarks on TryHackMe

Task	Category	Flag
Task 2	AS02 — Security Misconfiguration	THM{V3RB0S3_3RR0R_L34K}
Task 3	AS03 — Supply Chain Failure	THM{SUPPLY_CH41N_VULN3R4B1L1TY}
Task 4	AS04 — Cryptographic Failure	THM{CRYPTO_FAILURE_H4RDC0D3D_K3Y}

Task 5	AS06 — Insecure Design	THM{1NS3CUR3_D35IGN_4SSUMPT10N}
--------	------------------------	---------------------------------

Key Takeaways

- **AS02:** Disable debug mode in production. Never expose stack traces or internal variables in error responses.
- **AS03:** Audit all third-party libraries. Outdated or unverified dependencies can contain hidden backdoors.
- **AS04:** Never hard-code secrets in client-side code. Use strong encryption modes (AES-GCM not ECB) and store keys server-side.
- **AS06:** Design authentication and authorisation in from day one. A flawed design cannot be patched — it must be rebuilt.

Room completed ✓ — Continue with Room 3: Application Design Flaws on TryHackMe.